

파이썬 데이터 분석 기초 핵심 복습

키워드: 범주형/수치형 · 리스트 · 딕셔너리 · 시리즈 · 데이터프레임

작성일: 2026-03-21

이 노트는 *한 번에 훑어보는 복습용 치트시트 + 실행 가능한 예제 + 연습문제*로 구성되어 있습니다.

목차

- [데이터 타입: 범주형 vs 수치형](#1-데이터-타입-범주형-vs-수치형)
- [리스트(List)](#2-리스트list)
- [딕셔너리(Dictionary)](#3-딕셔너리dictionary)
- [Pandas 시리즈(Series)](#4-pandas-시리즈series)
- [Pandas 데이터프레임(DataFrame)](#5-pandas-데이터프레임dataframe)
- [한 장 요약(치트시트)](#6-한-장-요약치트시트)
- [연습문제 + 정답](#7-연습문제--정답)

1. 데이터 타입: 범주형 vs 수치형

데이터 분석의 첫 단계는 내가 다루는 값이 ‘수치’ 인지 ‘범주’ 인지 구분하는 것입니다.

정의

- 수치형(Numerical)**: 크기·거리 개념이 있고, 사칙연산이 의미 있음 (예: 키, 가격, 온도)
- 범주형(Categorical)**: 그룹/레이블을 나타냄. 평균 같은 연산이 직접적으로 의미 없을 수 있음 (예: 성별, 지역, 등급)

자주 쓰는 세부 분류

대분류	세부분류	설명	예시
수치형	연속형(continuous)	소수점 포함 가능, 연속적인 측정값	키 175.5, 온도 36.5
수치형	이산형(discrete)	보통 정수로 ‘개수’ 의미	자녀 수, 구매 횟수
범주형	명목형(nominal)	순서 없음	혈액형, 지역
범주형	순서형(ordinal)	순서/등급 있음	만족도(상/중/하), 학점(A/B/C)

왜 구분하나요?

- 통계**: 수치형은 평균/분산/회귀가 자연스럽고, 범주형은 빈도/비율/교차표가 핵심
- 시각화**: 수치형은 히스토그램/박스플롯, 범주형은 막대그래프가 기본
- 전처리**:
 - 범주형 → 인코딩(원-핫/라벨)
 - 수치형 → 스케일링(표준화/정규화)

대표 처리(개념)

- **라벨 인코딩(Label Encoding)**: 카테고리를 정수로 매핑 (순서형에 상대적으로 적합)
- **원-핫 인코딩(One-Hot Encoding)**: 카테고리마다 0/1 컬럼 생성 (명목형에 자주 사용)
- **표준화(Standardization)**: 평균 0, 표준편차 1로 변환 (스케일 차이 줄이기)
- **정규화(Normalization)**: 값 범위를 0~1로 맞춤 (거리 기반 모델에서 자주 등장)

자주 하는 실수/주의점

- 명목형에 라벨 인코딩만 하고 **정수 크기 의미가 생기는 착시**를 만들기
- 수치형처럼 보이지만 사실은 ID(예: 고객번호)인 경우 → **범주형처럼** 다루는 게 자연스러울 수 있음

핵심 포인트(3줄)

- 1) 수치형은 ‘계산’ 이 의미 있고, 범주형은 ‘분류’ 가 핵심
- 2) 범주형은 보통 인코딩, 수치형은 스케일링을 고려
- 3) 숫자로 되어 있다고 무조건 수치형은 아니다(ID 주의)

```
import pandas as pd
import numpy as np

# 실무 팁: dtype 확인과 변환
example = pd.DataFrame({
    'age': ['20', '21', 'not_available'],
    'gender': ['F', 'M', 'F'],
    'score': [88.5, 92.0, np.nan]
})

example
```

```
# dtype 확인
example.dtypes
```

```
# 숫자로 변환(to_numeric): 변환 불가 값은 NaN 처리 가능
example['age_num'] = pd.to_numeric(example['age'], errors='coerce')

# 범주형으로 명시(카테고리 dtype)
example['gender_cat'] = example['gender'].astype('category')

example
```

```
example.dtypes
```

2. 리스트(List)

리스트는 **순서가 있는 가변(mutable) 컨테이너**입니다.

핵심

- 생성: `[]`, `list()`
- 접근: 인덱싱 `a[0]`, 슬라이싱 `a[1:3]`
- 수정/추가/삭제가 자유로움

자주 쓰는 메서드

메서드	의미
<code>append(x)</code>	맨 끝에 추가
<code>extend(iterable)</code>	여러 개를 이어붙이기
<code>insert(i, x)</code>	i 위치에 삽입
<code>remove(x)</code>	값 x(첫 번째) 삭제
<code>pop(i)</code>	i 위치를 꺼내며 삭제(기본: 마지막)
<code>sort()</code> / <code>sorted(a)</code>	정렬(원본 변경 / 새 리스트)

얕은 복사 vs 깊은 복사(중요)

- `b = a` 는 복사 아님(같은 객체 참조)
- `b = a.copy()` 또는 `b = a[:]` 는 얕은 복사
- 중첩 리스트는 `copy.deepcopy(a)` 고려

핵심 포인트(3줄)

- 1) 리스트는 ‘순서 + 수정 가능’ 이 핵심
- 2) `sorted()` 는 새 리스트, `sort()` 는 원본 변경
- 3) 복사할 때 `b=a` 는 위험(같이 바뀜)

```
# 예제 1) 생성/인덱싱/슬라이싱
fruits = ['apple', 'banana', 'cherry']
print(fruits[1])      # banana
print(fruits[0:2])    # ['apple', 'banana']
```

```
# 예제 2) 추가/삭제
nums = [1, 2, 3]
nums.append(4)
nums.insert(1, 10)
nums.remove(3)
last = nums.pop()
nums, last
```

```
# 예제 3) 얕은 복사 이슈
a = [[1, 2], [3, 4]]
b = a.copy() # 얕은 복사
b[0][0] = 999
a, b
```

3. 딕셔너리(Dictionary)

딕셔너리는 **Key-Value** 구조로, 키로 값을 빠르게 조회하는 자료구조입니다.

핵심

- 생성: `{}`, `dict()`
- 키는 보통 문자열을 많이 쓰며 중복 불가
- `in` 으로 키 존재 여부 확인 가능

자주 쓰는 메서드

--	--

메서드	의미
<code>get(k, default)</code>	키 없을 때도 안전하게 조회
<code>keys()</code>	키 목록
<code>values()</code>	값 목록
<code>items()</code>	(키, 값) 쌍

Dict comprehension

- 패턴: `{k: f(v) for k, v in ...}`

핵심 포인트(3줄)

- 1) 딕셔너리는 ‘키로 빠르게 찾기’에 최적
- 2) `get`을 쓰면 `KeyError`를 줄일 수 있음
- 3) 카운팅/매핑/룩업 테이블에 자주 사용

```
# 기본 예제
scores = {'math': 90, 'eng': 85, 'kor': 95}

scores['math'], scores.get('sci', 0)
```

```
# 실전 예제: 카운팅(빈도)
words = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
counts
```

```
# dict comprehension으로 변환/가공
price = {'apple': 1000, 'banana': 800, 'orange': 1200}
price_vat = {k: int(v * 1.1) for k, v in price.items()} # 부가세 10%
price_vat
```

4. Pandas 시리즈(Series)

Series는 인덱스(index)를 가진 1차원 데이터입니다. (엑셀의 ‘한 열’ 느낌)

핵심

- `pd.Series(data, index=...)`
- `dtype`, `isna()`, boolean indexing(조건 필터)

핵심 포인트(3줄)

- 1) Series는 1차원 + index 라벨이 특징
- 2) 결측치(NaN) 처리 흐름을 익히면 실무가 편해짐
- 3) 조건 필터(Boolean indexing)는 가장 많이 쓰는 패턴

```
import pandas as pd
import numpy as np
```

```
s = pd.Series([10, 20, np.nan, 40], index=['a', 'b', 'c', 'd'])
s
```

```
# dtype / 결측치
s.dtype, s.isna()
```

```
# 결측치 처리 예
s_filled = s.fillna(0)
s_dropped = s.dropna()

s_filled, s_dropped
```

```
# boolean indexing
s[s >= 20]
```

5. Pandas 데이터프레임(DataFrame)

DataFrame은 **행(row)**과 **열(column)**로 구성된 2차원 테이블입니다.

핵심 선택/필터 패턴

- 컬럼 선택: `df['col']`, `df[['c1', 'c2']]`
- 행/열 선택:
 - `loc`: 라벨 기반
 - `iloc`: 위치 기반
- 조건 필터: `df[df['Age'] >= 25]`

집계/그룹

- `groupby`로 범주 기준 집계

병합

- `merge`로 키 기준 결합(SQL join 느낌)

핵심 포인트(3줄)

- DataFrame은 실무 분석의 기본 단위(엑셀/테이블)
- `loc/iloc`과 조건 필터는 반드시 익혀야 함
- `groupby/merge`가 되면 ‘분석다운 분석’이 가능해짐

```
# 예제 데이터
import pandas as pd

df = pd.DataFrame({
    'Name': ['Kim', 'Lee', 'Park', 'Choi'],
    'Age': [25, 30, 22, 30],
    'City': ['Seoul', 'Busan', 'Incheon', 'Seoul'],
    'Spend': [12000, 18000, 7000, 22000]
})

df
```

```
# 기본 정보 확인
(df.shape, df.dtypes)
```

```
# 선택: 컬럼
names = df['Name']
subset = df[['Name', 'City']]
names, subset
```

```
# 선택: loc / iloc
first_two_rows = df.iloc[:2]
seoul_rows = df.loc[df['City'] == 'Seoul', ['Name', 'Spend']]

first_two_rows, seoul_rows
```

```
# 조건 필터 + 정렬
result = df[df['Age'] >= 25].sort_values('Spend', ascending=False)
result
```

```
# groupby 집계: 도시별 평균 지출
city_stats = df.groupby('City', as_index=False).agg(
    n=('Name', 'count'),
    avg_spend=('Spend', 'mean'),
    max_spend=('Spend', 'max')
)
city_stats
```

```
# merge(병합) 예제
city_region = pd.DataFrame({
    'City': ['Seoul', 'Busan', 'Incheon'],
    'Region': ['Capital', 'Yeongnam', 'Sudogwon']
})

merged = df.merge(city_region, on='City', how='left')
merged
```

6. 한 장 요약(치트시트)

자료구조 비교

키워드	형태	핵심 특징	언제 쓰나
리스트	[]	순서 있음, 수정 가능	순서가 있는 값 모음
딕셔너리	{k: v}	키로 빠른 조회	매핑/카운팅/룩업
Series	1차원 + index	결측치/필터링에 강함	단일 컬럼 분석
DataFrame	2차원 테이블	선택/집계/병합	데이터 분석의 기본

Pandas 실무 팁(타입)

- 타입 확인: `df.dtypes`
- 변환: `astype`, `pd.to_numeric`, `pd.to_datetime`
- 범주형 최적화: `astype('category')`

다음 학습 추천

- 결측치 처리 전략(단순 대체 vs 모델 기반)
- 피처 엔지니어링(날짜 파생, 범주 조합)

- 3) 시각화(matplotlib/seaborn) 기본 패턴
- 4) 통계 기초(표본/분포/가설검정)
- 5) 머신러닝 전처리 파이프라인(sklearn Pipeline)

7. 연습문제 + 정답

문제(10개)

- 1) 다음 중 범주형(순서형) 데이터는? (키 / 강수량 / 별점 1~5 / 좋아하는 과일)
- 2) 리스트 `a=[1,2,3]` 에서 마지막 값을 꺼내며 삭제하는 메서드는?
- 3) `sorted(a)` 와 `a.sort()` 의 차이를 한 줄로 설명하세요.
- 4) 딕셔너리에서 키가 없을 때도 안전하게 값을 가져오는 메서드는?
- 5) 단어 리스트 `['a','b','a']` 의 빈도를 딕셔너리로 세어보세요.
- 6) Series에서 결측치 여부를 True/False로 반환하는 메서드는?
- 7) DataFrame에서 상위 3행을 보는 메서드는?
- 8) `df.loc` 와 `df.iloc` 의 차이는?
- 9) City 별 Spend 평균을 구하는 groupby 코드를 작성하세요.
- 10) City 컬럼을 키로 df 와 city_region 을 left join하는 merge 코드를 작성하세요.

정답(간단)

- 1) 별점 1~5
- 2) `pop()`
- 3) `sorted` 는 새 리스트 반환, `sort` 는 원본을 정렬(제자리)
- 4) `get`
- 5) 아래 코드 셀 참고
- 6) `isna()` 또는 `isnull()`
- 7) `head(3)`
- 8) loc=라벨 기반, iloc=위치(정수) 기반
- 9) 아래 코드 셀 참고
- 10) 아래 코드 셀 참고

```
# 5) 빈도 세기 정답 예시
words = ['a', 'b', 'a']
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
counts
```

```
# 9) City별 Spend 평균(groupby)
df.groupby('City')['Spend'].mean()
```

```
# 10) merge(left join)
city_region = pd.DataFrame({'City': ['Seoul', 'Busan', 'Incheon'], 'Region': ['Capital', 'Yeongnam', 'Sudogwon']})

df.merge(city_region, on='City', how='left')
```